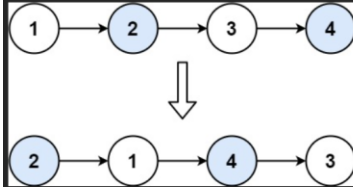


24. 两两交换链表中的节点

中等 相关标签 相关企业 42

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在`不修改节点内部的值`的情况下完成本题（即，只能进行节点交换）。

示例 1:



输入: head = [1,2,3,4]
输出: [2,1,4,3]

示例 2:

输入: head = []
输出: []

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* swapPairs(ListNode* head) {
14         ListNode* dummy = new ListNode(0);
15         dummy->next = head;
16         ListNode* cur = dummy;
17         while(cur->next!=nullptr && cur->next->next!=nullptr){
18             ListNode* node1 = cur->next;
19             ListNode* node2 = cur->next->next;
20             cur->next = node2;
21             node1->next = node2->next;
22             node2->next = node1;
23             cur = node1;
24         }
25         ListNode* ans = dummy->next;
26         delete dummy;
27         return ans;
28     }
29 };

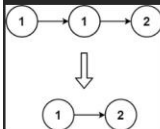
```

83. 删除排序链表中的重复元素

简单 相关标签 相关企业 42

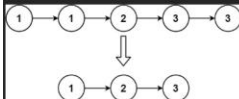
给定一个已排序的链表的头 `head`，删除所有重复的元素，使每个元素只出现一次。返回已排序的链表。

示例 1:



输入: head = [1,1,2]
输出: [1,2]

示例 2:



输入: head = [1,1,2,3,3]
输出: [1,2,3]

提示:

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* deleteDuplicates(ListNode* head) {
14         //用哈希表存储元素 key值 value出现次数
15         unordered_map<int,int> mp;
16         //遍历链表
17         ListNode* dummy = new ListNode(0);
18         dummy->next = head;
19         ListNode* pre = dummy;
20         ListNode* curr = dummy->next;
21         //遍历
22         while(curr){
23             //如果在哈希表中找到了当前元素 说明当前元素之前出现过 删除当前节点
24             if(mp.find(curr->val)!=mp.end()){
25                 ListNode* delNextNode = curr->next;
26                 pre->next = delNextNode;
27                 curr = curr->next;
28             }else{
29                 mp[curr->val] += 1;
30                 pre = curr;
31                 curr = curr->next;
32             }
33         }
34         ListNode* ans = dummy->next;
35         delete dummy;
36         return ans;
37     }
38 };

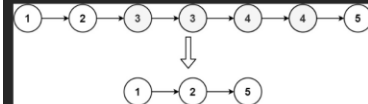
```

82. 删除排序链表中的重复元素 II

中等 相关标签 相关企业 42

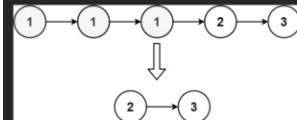
给定一个已排序的链表的头 `head`，删除原始链表中所有重复数字的节点，只留下不同的数字。返回已排序的链表。

示例 1:



输入: head = [1,2,3,3,4,4,5]
输出: [1,2,5]

示例 2:



输入: head = [1,1,1,2,3]
输出: [2,3]

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* deleteDuplicates(ListNode* head) {
14         //用一个哈希表，记录每个数字出现次数 之后再遍历一次链表 如果哈希表中出现次数大于1就删除此节点
15         unordered_map<int,int> mp;
16         //遍历链表 存入元素
17         ListNode* cur = head;
18         while(cur){
19             mp[cur->val]++;
20             cur = cur->next;
21         }
22         //再次遍历链表
23         ListNode* dummy = new ListNode(0);
24         dummy->next = head;
25         cur = dummy->next;
26         ListNode* pre = dummy;
27         while(cur){
28             //如果当前节点出现次数大于1 删除此节点
29             if(mp[cur->val]>1){
30                 pre->next = cur->next;
31                 cur = cur->next;
32             }else{
33                 pre = cur;
34                 cur = cur->next;
35             }
36         }
37         ListNode* ans = dummy->next;
38         delete dummy;
39         return ans;
40     }
41 };

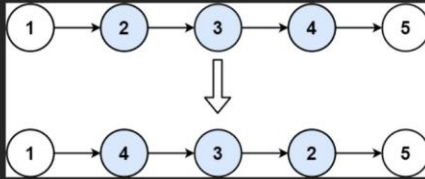
```

92. 反转链表 II

中等 相关标签 相关企业 Az

给你单链表的头指针 `head` 和两个整数 `left` 和 `right`，其中 `left <= right`。请你反转从位置 `left` 到位置 `right` 的链表节点，返回 反转后的链表。

示例 1:



输入: `head = [1,2,3,4,5]`, `left = 2`, `right = 4`
输出: `[1,4,3,2,5]`

示例 2:

输入: `head = [5]`, `left = 1`, `right = 1`
输出: `[5]`

提示:

- 链表中节点数目为 `n`
- $1 \leq n \leq 500$

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* reverseBetween(ListNode* head, int left, int right) {
14         stack<ListNode*> st;
15         ListNode* dummy = new ListNode(0);
16         dummy->next = head;
17         ListNode* pre = dummy;
18
19         // 第一步: pre 走到反转区间的前驱节点 (走 left-1 步)
20         for(int i = 0; i < left - 1; i++){
21             pre = pre->next;
22         }
23
24         // 第二步: 把反转区间的所有节点依次入栈
25         ListNode* cur = pre->next;
26         for(int i = 0; i < right - left + 1; i++){
27             st.push(cur);
28             cur = cur->next;
29         }
30         // 此时 cur 指向反转区间后面的第一个节点
31
32         // 第三步: 弹栈拼接, 完成反转
33         while(!st.empty()){
34             pre->next = st.top();
35             st.pop();
36             pre = pre->next;
37         }
38     }
39 };
```

3471. 找出最大的几近缺失整数

简单 相关标签 相关企业 提示 Az

给你一个整数数组 `nums` 和一个整数 `k`。

如果整数 `x` 恰好仅出现在 `nums` 中的一个大小为 `k` 的子数组中，则认为 `x` 是 `nums` 中的几近缺失 (almost missing) 整数。

返回 `nums` 中 最大的几近缺失 整数，如果不存在这样的整数，返回 `-1`。

子数组 是数组中的一个连续元素序列。

示例 1:

输入: `nums = [3,9,2,1,7]`, `k = 3`

输出: 7

解释:

- 1 出现在两个大小为 3 的子数组中: `[9, 2, 1]`、`[2, 1, 7]`
- 2 出现在三个大小为 3 的子数组中: `[3, 9, 2]`、`[9, 2, 1]`、`[2, 1, 7]`
- 3 出现在一个大小为 3 的子数组中: `[3, 9, 2]`

C++ 智能模式

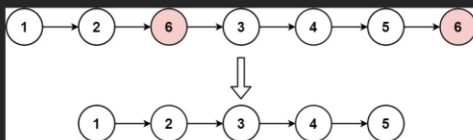
```
1 class Solution {
2 public:
3     int largestInteger(vector<int>& nums, int k) {
4         // 一个数组中 重复出现的元素不统计
5         unordered_map<int,int> mp(0); // 记录某个数出现几次
6         int len = nums.size();
7         int ans = INT_MIN;
8         for(int i = 0; i < len - k + 1; i++){
9             unordered_map<int,int> mp1(0); // 在这个循环里 某元素是否多次出现
10            for(int j = i; j < i + k; j++){
11                mp1[nums[j]]++;
12                if(mp1[nums[j]] <= 1){
13                    mp[nums[j]]++;
14                }
15            }
16        }
17        bool haveOne = false;
18        for(auto num: nums){
19            if(mp[num] == 1){
20                haveOne = true;
21                ans = max(ans, num);
22            }
23        }
24        return haveOne ? ans : -1;
25    }
26 };
27 ;;
```

203. 移除链表元素

简单 相关标签 相关企业 Az

给你一个链表的头节点 `head` 和一个整数 `val`，请你删除链表中所有满足 `Node.val == val` 的节点，并返回 新的头节点。

示例 1:



输入: `head = [1,2,6,3,4,5,6]`, `val = 6`
输出: `[1,2,3,4,5]`

示例 2:

输入: `head = []`, `val = 1`
输出: `[]`

示例 3:

C++ 智能模式

```
1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode() : val(0), next(nullptr) {}
7  *     ListNode(int x) : val(x), next(nullptr) {}
8  *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9  * };
10 */
11 class Solution {
12 public:
13     ListNode* removeElements(ListNode* head, int val) {
14         ListNode* dummy = new ListNode;
15         dummy->next = head;
16         ListNode* cur = dummy;
17         while(cur != nullptr && cur->next != nullptr){
18             if(cur->next->val == val){
19                 cur->next = cur->next->next;
20             }else{
21                 cur = cur->next;
22             }
23         }
24         ListNode* ans;
25         ans = dummy->next;
26         delete dummy;
27         return ans;
28     }
29 };
```

LCR 025. 两数相加 II

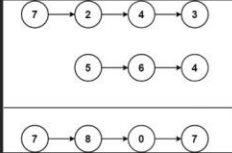
已解答

中等 相关标签 相关企业

给定两个非空链表 `l1` 和 `l2` 来代表两个非负整数。数字最高位位于链表开始位置。它们的每个节点只存储一位数字。将这两数相加会返回一个新的链表。

可以假设除了数字 0 之外，这两个数字都不会以零开头。

示例 1:



输入: `l1 = [7,2,4,3]`, `l2 = [5,6,4]`
输出: `[7,8,0,7]`

示例 2:

输入: `l1 = [2,4,3]`, `l2 = [5,6,4]`
输出: `[8,0,7]`

示例 3:

输入: `l1 = [0]`, `l2 = [0]`
输出: `[0]`

提示:

- 链表的长度范围为 `[1, 100]`
- `0 <= node.val <= 9`
- 输入数据保证链表代表的数字无前导 0

```
C++ 智能模式
1  ListNode* : val(w), next(nullptr) {}
2  *
3  ListNode(int x) : val(x), next(nullptr) {}
4  *
5  ListNode(int x, ListNode *next) : val(x), next(next) {}
6  *
7  *
8  *
9  *
10 *
11
12 class Solution {
13 public:
14     ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
15         // 用栈实现
16         stack<int> st1, st2;
17         // 入栈
18         while(l1){
19             st1.push(l1->val);
20             l1 = l1->next;
21         }
22         while(l2){
23             st2.push(l2->val);
24             l2 = l2->next;
25         }
26         // 开始模拟加法
27         int carry = 0; // 进位
28         ListNode* ans = nullptr;
29         // 只要两个栈或进位都不是空的就循环
30         while(!st1.empty() || !st2.empty() || carry != 0){
31             // 模拟加法 先取低位数字
32             int num1 = st1.empty() ? 0 : st1.top();
33             int num2 = st2.empty() ? 0 : st2.top();
34             if(!st1.empty()) st1.pop();
35             if(!st2.empty()) st2.pop();
36             // 计算
37             int digitSum = num1 + num2 + carry;
38             // 当前位置进位数字
39             carry = digitSum / 10;
40             // 当前位置数字
41             digitSum %= 10;
42             // 为当前位置创建新结点
43             auto curNode = new ListNode(digitSum);
44             // 头插到结果链表
45             curNode->next = ans;
46             // 更新ans位置 始终指向第一个结点
47             ans = curNode;
48         }
49         return ans;
50     };
51 }
```